MAKERERE UNIVERSITY

College of Computing and Information Sciences

School of Computing and Informatics Technology, Department of Networks

BSE4104: Emerging Trends in Software Engineering

RESOLV-HQ

Autonomous Service Desk & Ticket Dispatcher

WEEK 1 SUBMISSION PACKAGE

Problem Framing and AI-Native Requirements

Week Ending: 4th September 2026

Contents

1. Minimum Proposal Statement

2. Project Charter

3. AI Boundary Matrix

4. User Stories & Acceptance Criteria

5. Initial System Architecture

6. Week 1 Progress Report

GROUP H

NAME	REG NUMBER
Karungi Irene	23/U/09080/EVE
Drate Hillary	23/U/23611
YAWE IVAN	23/U/18666/EVE
Opiro Adrine	23/U/16892/EVE

Contents

1. Minimum Proposal Statement 3
2. Project Charter 3
2.1 Executive Summary 3
2.2 Problem Framing & Target User Pain Points 3
2.3 Strategic AI Justification (Why AI/RAG/Tools vs. Deterministic Rules) 4
2.4 High-Level System Workflow 4
2.5 Project Assumptions, Constraints & Risks 5
Assumptions 5
Constraints 5
Top Risks (summary — full register in Section 6.3) 5
3. AI Boundary Matrix 6
4. User Stories & Acceptance Criteria 7
User Story 1. Service Client 7
User Story 2. Service Client 7
User Story 3. Support Agent 7
User Story 4. Support Agent 7
User Story 5. Service Client 8
User Story 6. Team Lead 8
User Story 7. System Administrator 8
User Story 8. QA & Security Lead 9
5. Initial System Architecture 9
6. Week 1 Progress Report 11
6.1 Objectives vs. Achievements 11
6.2 Key Engineering Decisions & Rationale 11
6.3 Risk Register & Mitigations 11
6.4 Individual Contribution Matrix 12
6.5 GitHub Repository Structure 13
6.6 Plan for Next Week (Week 2) 13

1. Minimum Proposal Statement

Our system helps **enterprise customer support agents and their service clients** complete **the triage, diagnosis, and dispatch of customer support tickets**. AI is used for **reasoning over the customer's stated issue, retrieval of grounded policy and troubleshooting knowledge, and planning the next diagnostic or dispatch action**. Deterministic software remains responsible for **authentication, authorization, business-rule validation, and execution of any financial or account-state change**. The agent may use **approved tools policy/knowledge RAG search, account-status lookup, outage-status checker, and a draft-ticket/escalation-summary generator** but may not **authorize refunds, alter billing or account-security settings, or close/resolve a ticket without human sign-off**. We will build and evaluate the system using **a synthetic, team-authored customer-support corpus comprising policy documents, FAQ articles, a mock customer/account database, and a simulated service-outage log**.

2. Project Charter

2.1 Executive Summary

RESOLV-HQ is a bounded, agentic customer-support and ticket-routing system designed for enterprise service desks that handle a high volume of repetitive account, billing, and technical-outage inquiries. Rather than replacing human support staff, RESOLV-HQ acts as a first-line reasoning layer: it interprets a customer's natural-language issue, grounds its understanding of company policy in a retrieval-augmented knowledge base, calls read-only diagnostic tools to check account and outage status, and either resolves simple, low-risk requests directly or drafts a structured escalation summary for a human agent. Every action with financial, security, or account-state consequences is deliberately withheld from the agent and routed through a human-approval checkpoint. This charter defines the problem, the target users, the justification for using AI rather than a purely deterministic system, the high-level workflow, and the assumptions, constraints, and risks that will guide the first iteration of the system.

2.2 Problem Framing & Target User Pain Points

Enterprise support teams are structurally overloaded by ticket volume that is disproportionately repetitive. Service clients, meanwhile, experience long queues for problems that are often already documented or already known to the organisation (e.g., an ongoing outage). Two existing approaches fall short:

- Generic chatbots can converse fluently but have no access to live account data or diagnostic tools, and they frequently hallucinate policy details because they rely on parametric memory rather than the organisation's actual, current policy documents.
- Rule-based engines (decision trees, keyword routers) are predictable and auditable but cannot interpret free-form, ambiguous, or multi-intent customer language, and they require constant manual maintenance as policies and product lines change.

The specific pain points RESOLV-HQ targets are:

- Support agents: spend significant time re-reading long, unstructured conversation transcripts before they can act on an escalated ticket.
- Service clients: wait in queue for questions that are answerable from existing policy documents or that require only an account-status check.
- Support teams (management): lack a consistent, auditable trail of why a ticket was answered, escalated, or held for approval.

- Compliance/Quality staff: need assurance that no AI-authored response has quietly authorised a refund, a plan change, or another financially or legally significant action.

2.3 Strategic AI Justification (Why AI/RAG/Tools vs. Deterministic Rules)

A purely deterministic system (a rule engine or fixed decision tree) is not viable as the sole interface because customer language is unstructured, ambiguous, and often bundles multiple intents in a single message (e.g., "my internet has been down since this morning and I was also charged twice"). Encoding this variability into hand-written rules does not scale and becomes brittle whenever a policy changes. A generic large language model without retrieval or tools is equally unsuitable because it cannot verify current account state, cannot confirm an active outage, and critically will produce fluent but unverifiable answers about refund policy, SLA terms, or account eligibility, which is an unacceptable hallucination risk in a support context.

RESOLV-HQ therefore combines the strengths of each approach rather than choosing one exclusively:

- Retrieval-Augmented Generation (RAG) grounds every policy-related answer in the organisation's actual, versioned policy corpus, so the agent cites what the policy says rather than recalling it from training data.
- Bounded tool use gives the agent read access to live diagnostic data (account status, outage status) so its reasoning reflects the customer's actual situation, not a generic assumption.
- Deterministic software remains the sole authority over authentication, authorization, and any state-changing or financial action, so the unpredictability of a language model is confined to reasoning, drafting, and retrieval — never to execution of consequential actions.

This hybrid division of labour is the central engineering thesis of the project: AI is used only where its flexibility adds value (understanding intent, retrieving and summarising relevant policy, drafting a structured escalation), and deterministic control is retained wherever an error would be costly, irreversible, or would require legal/financial accountability.

2.4 High-Level System Workflow

1. A service client submits a support request in natural language through the front-end interface.
2. Deterministic middleware authenticates the client and attaches their verified account identifier to the session; the agent never receives raw credentials.
3. The ReAct-style agent core interprets the request and decides on a next action: retrieve policy context, call a diagnostic tool, ask a clarifying question, or draft a response.
4. The RAG engine retrieves the most relevant policy/FAQ passages from the vector knowledge base and returns them with source references.
5. If account-specific or outage-specific information is required, the agent calls the relevant read-only tool (account-status lookup or outage-status checker) against the mock CSV/database backend.
6. For low-risk, fully answerable requests (e.g., a documented FAQ, a confirmed known outage), the agent drafts a direct response, which deterministic software logs and returns to the client.
7. For any request involving ambiguity, dissatisfaction, or a higher-impact action (refunds, account changes, unresolved technical faults), the agent drafts a structured escalation ticket summary, evidence retrieved, suggested next step and routes it to the Human Admin Approval Dashboard.
8. A human agent reviews, edits if necessary, and approves or rejects the drafted action before any escalation, refund, or account change is executed; deterministic software then performs the approved action and closes the loop back to the client.

2.5 Project Assumptions, Constraints & Risks

Assumptions

- The team will use synthetic, self-authored data (mock customers, mock tickets, mock policy documents) for all seven weeks, since no live enterprise support data or authorization to use it currently exists.
- A single support domain (a fictional ISP/telecom-style service provider) is sufficient to demonstrate the full agentic pipeline without introducing unnecessary domain complexity.
- Team members have access to a foundation-model API (to be finalised in Week 2) and to standard Python/JavaScript tooling for RAG and orchestration.

Constraints

- The eight-week timeline limits the system to one primary end-to-end workflow (ticket triage and dispatch) rather than a full omni-channel support platform.
- No real financial transactions, real customer PII, or production account systems may be touched; all "accounts" and "payments" are simulated.
- The agent's action space is intentionally small in Week 1–4 (read-only tools only) to keep the system explainable and testable before bounded autonomy is introduced in Week 5.

Top Risks (summary, full register in Section 6.3)

- Hallucinated policy answers if RAG retrieval is weak or the corpus is too sparse.
- Scope creep toward a full CRM rather than the single triage-and-dispatch workflow.
- Over-permissioning the agent (accidentally allowing a state-changing action) before human-approval controls are implemented.

3. AI Boundary Matrix

The matrix below defines, for every major system action, whether control sits with deterministic software, the bounded AI agent, or requires explicit human approval before execution. This matrix is the authoritative safety boundary for RESOLV-HQ and will be re-validated at the end of every subsequent week.

Capability / System Action	**Deterministic Software**	**Bounded AI Agent**	**Explicit Human Approval Required**
Authentication & Session Verification	Full control validates credentials, issues session token	No access to raw credentials	Not required (system-level, non-AI)
Policy & Knowledge Retrieval (RAG)	Enforces which knowledge sources are indexed and versioned	Retrieves and synthesises grounded answers with citations	Not required for read-only retrieval
Account Status Lookup (read-only tool)	Executes the query; agent cannot write to the database	Decides when to call the tool and interprets the result	Not required read-only, non-destructive
Outage / Service Status Check (read-only tool)	Serves authoritative outage records	Decides when to call the tool and explains impact to the client	Not required read-only, non-destructive
Draft Response to Simple/FAQ Query	Logs and delivers the final message	Drafts the response text grounded in retrieved policy	Spot-audited, not gated per response
Escalation Ticket Creation & Routing	Persists the ticket record and routing metadata	Drafts the structured summary, priority, and suggested owner	Required before the ticket is dispatched to a live queue
Financial Refunds / Billing Adjustments	Executes the transaction only after approval flag is set	May recommend a refund with justification only	Required no refund executes without a human's explicit sign-off
Account Changes (plan, security, contact info)	Executes the change only after approval flag is set	May draft the proposed change for review	Required no account mutation without human approval
Conversation & Decision Audit Logging	Immutable, system-level logging of every agent action/tool call	No control over what is logged or retained	Not required; logs are reviewable on demand by QA/Security

Table 1. AI Boundary Matrix for RESOLV-HQ (Week 1 baseline).

4. User Stories & Acceptance Criteria

The following eight user stories define the testable scope of the Week 1–4 increments. Each story includes three Given/When/Then acceptance criteria.

User Story 1. Service Client

As a Service Client, I want to describe my problem in plain language so that I do not need to navigate a rigid support menu.

Acceptance Criteria:

- Given a client opens a new support session, When they type a free-text description of their issue, Then the agent produces an initial interpretation of the intent within one turn.
- Given the client's message contains more than one issue, When the agent processes it, Then the agent identifies and lists each distinct issue separately.
- Given the client's message is too vague to act on, When the agent cannot determine intent, Then the agent asks exactly one targeted clarifying question rather than guessing.

User Story 2. Service Client

As a Service Client, I want the agent to check my account status automatically so that I do not have to re-enter details already on file.

Acceptance Criteria:

- Given a client is authenticated, When their issue requires account context, Then the agent calls the account-status tool without asking the client to repeat account details.
- Given the account-status tool returns a result, When the agent responds, Then the response reflects the retrieved account data accurately.
- Given the account-status tool is unavailable, When the agent attempts the call, Then the agent informs the client of the temporary limitation instead of fabricating account data.

User Story 3. Support Agent

As a Support Agent, I want the agent to show the exact policy passage it used so that I can verify accuracy before a response reaches the customer.

Acceptance Criteria:

- Given the agent answers a policy-related question, When the response is generated, Then it includes a citation to the specific source document and section retrieved.
- Given no relevant policy passage is retrieved, When the agent would otherwise answer, Then it explicitly states that no grounded source was found rather than answering unsupported.
- Given a Support Agent opens the trace view, When they inspect a past answer, Then they can see the exact retrieved passages that were passed to the model.

User Story 4. Support Agent

As a Support Agent, I want complex or ambiguous tickets routed to me with a structured summary so that I can act quickly without re-reading the full transcript.

Acceptance Criteria:

- Given the agent classifies a ticket as complex or ambiguous, When it escalates, Then it produces a structured summary containing issue, evidence retrieved, and a suggested next step.
- Given an escalation is created, When it appears on the Human Admin Approval Dashboard, Then it is tagged with a priority level and the originating client identifier.
- Given a Support Agent opens an escalated ticket, When they review it, Then they can see the full underlying conversation on request, not only the summary.

User Story 5. Service Client

As a Service Client, I want to be told about a known outage so that I do not file a duplicate ticket for an issue already being fixed.

Acceptance Criteria:

- Given an active outage record matches the client's reported symptom and location, When the agent responds, Then it informs the client of the known outage and its status.
- Given no outage record matches, When the client reports a technical fault, Then the agent proceeds with normal diagnostic and escalation steps.
- Given an outage is already known, When the agent responds, Then it does not create a duplicate escalation ticket for the same outage.

User Story 6. Team Lead

As a Team Lead, I want any refund or account-change request to require my explicit approval so that no financial or account action happens without oversight.

Acceptance Criteria:

- Given the agent identifies a refund-eligible situation, When it responds, Then it drafts a recommendation only and does not execute a transaction.
- Given a drafted refund or account-change recommendation exists, When it reaches the approval dashboard, Then it cannot be executed until a human explicitly approves it.
- Given a Team Lead rejects a drafted recommendation, When rejection is recorded, Then the system logs the rejection reason and does not retry the same action automatically.

User Story 7. System Administrator

As a System Administrator, I want every AI action and tool call logged with timestamps so that I can audit agent behaviour and investigate errors.

Acceptance Criteria:

- Given the agent performs any reasoning step, tool call, or draft, When the action completes, Then it is recorded in an immutable log with a timestamp and session identifier.
- Given a System Administrator queries the log for a specific session, When they retrieve it, Then they see the full sequence of retrieved sources, tool calls, and drafted outputs in order.
- Given a tool call fails, When the failure occurs, Then the failure and the agent's fallback behaviour are both captured in the log.

User Story 8. QA & Security Lead

As a QA & Security Lead, I want the agent to refuse to fabricate a policy answer when retrieval returns no relevant match so that customers never receive hallucinated information.

Acceptance Criteria:

- Given the RAG engine returns zero passages above the relevance threshold, When the agent would answer a policy question, Then it declines to answer definitively and offers escalation instead.
- Given a QA tester submits a deliberately unanswerable question, When the agent responds, Then the response does not assert an unsupported fact.
- Given a fabricated or unsupported answer is detected during testing, When it is logged as a failure, Then it is added to the failure catalogue with the retrieval conditions that caused it.

5. Initial System Architecture

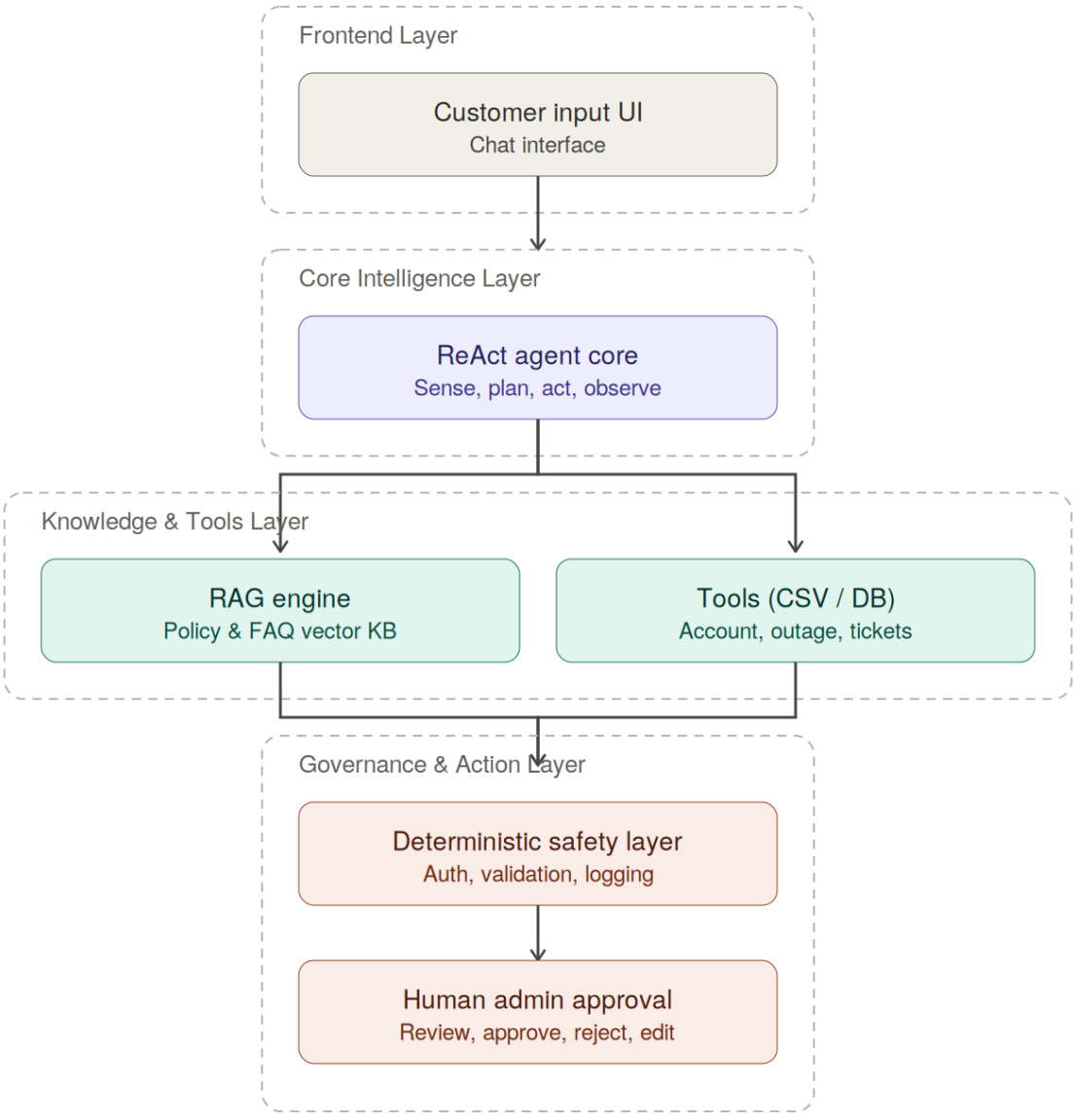


Figure 1. RESOLV-HQ initial system architecture.

The Customer Input / Frontend captures the client's natural-language request and forwards it, together with a verified session token issued by deterministic authentication middleware, to the ReAct Agent Core. The

agent core is the only component permitted to reason freely: on each turn it decides whether to retrieve policy context from the RAG Engine / Vector Knowledge Base, call one of the read-only diagnostic tools backed by the mock CSV/database layer, ask a clarifying question, or produce a final response. Both the RAG Engine and the Tools layer are strictly read-only and return evidence retrieved passages or lookup results rather than taking any action themselves. All agent output then passes through a Deterministic Safety Layer, which is the sole component authorised to log the interaction, validate the proposed action against the AI Boundary Matrix, and gate anything financial, account-altering, or otherwise high-risk. Low-risk, fully-grounded responses (e.g., an FAQ answer with a valid citation) are delivered directly to the client. Anything involving ambiguity, dissatisfaction, refunds, or account changes is routed instead to the Human Admin Approval Dashboard, where a human reviews, edits, approves, or rejects the drafted action before it is ever executed. This separation ensures the agent can reason and draft freely while remaining structurally unable to execute a consequential action on its own.

6. Week 1 Progress Report

Project: RESOLV-HQ — Autonomous Service Desk & Ticket Dispatcher | Course: BSE4104

Week Ending: 4th September 2026 | Report Prepared By: Project Lead

6.1 Objectives vs. Achievements

Week 1 Objective (per course brief)	Achievement / Status
Select one feasible use case and define a single primary end-to-end workflow	Completed. Selected: agentic customer support triage and ticket dispatch (RESOLV-HQ). Primary workflow: intake -> RAG/tool diagnosis -> auto-resolve or escalate -> human approval for high-risk actions.
Write a 2–3 page Project Charter (problem, user, pain point, AI value, scope, assumptions, constraints)	Completed and included in Section 2 of this package.
Create 8–12 testable user stories with acceptance criteria	Completed. Eight user stories drafted, each with three Given/When/Then acceptance criteria (Section 4).
Create an AI Boundary Matrix (AI vs. deterministic vs. human-approval)	Completed. Nine system actions classified (Section 3).
Produce an initial architecture / context diagram	Completed. ASCII diagram and component-interaction narrative (Section 5).
Create the GitHub repository and ClickUp project; assign Week 1 tasks	Completed. Repository initialised with the recommended structure (Section 6.5); ClickUp board created with Week 1 tasks assigned to all four members.

6.2 Key Engineering Decisions & Rationale

- Decision: Use a synthetic, team-authored CSV/database corpus (mock accounts, mock outage log, mock policy documents) instead of seeking a real enterprise dataset. Rationale: guarantees data availability and authorisation for the full eight weeks, avoids privacy/confidentiality concerns, and is explicitly permitted by the course brief for teams without prior data-sharing authorisation.
- Decision: Adopt a RAG-plus-tools architecture over a single large system prompt. Rationale: keeps policy knowledge separately versioned and auditable, and keeps read-only diagnostic capability separated from generative reasoning, which is required for the Week 3–4 RAG and tool-calling milestones and improves testability.
- Decision: Restrict the agent to read-only tool access in this phase, with all state-changing actions deferred to the Week 5 bounded-autonomy milestone. Rationale: allows the boundary matrix and safety layer to be validated before any irreversible action is technically possible, reducing risk during early iterations.
- Decision: Define escalation ticket creation as "draft-and-route" rather than "create-and-dispatch." Rationale: keeps a human decision point in the loop for every ticket that leaves the automated FAQ/status-check path, in line with the mandatory human-approval requirement for higher-impact actions.

6.3 Risk Register & Mitigations

Risk	Likelihood	Impact	Mitigation
Hallucinated or unsupported policy answers if the RAG corpus is too sparse or retrieval is weak	Medium	High	Enforce a minimum-relevance threshold below which the agent must decline to answer; expand the corpus in Week 3 with provenance-tracked documents; add explicit unanswerable test cases.
Scope creep toward a full CRM/omni-channel platform instead of the single triage-and-dispatch workflow	Medium	Medium	Charter explicitly scopes one primary workflow; weekly reports will flag any deviation for team review before implementation.
Accidental over-permissioning of the agent before the safety layer is fully implemented	Low	High	All tools are implemented as strictly read-only in this phase; state-changing actions are stubbed and disabled until human-approval gating exists (Week 4–5).
Uneven task ownership across the four-person team	Low	Medium	ClickUp board with individually assigned, deadline-bound tasks; each member documents evidence of their own contribution weekly.

6.4 Individual Contribution Matrix

Member / Role	Task(s) Owned This Week	Evidence
Project Lead	Use-case selection facilitation; authored Project Charter (Sections 2.1–2.5); consolidated and submitted the Week 1 package; created and managed the ClickUp board.	docs/weekly-reports/week1.md; ClickUp board link; charter draft commit history.
AI Engineering Lead	Drafted the Minimum Proposal Statement; designed the AI Boundary Matrix; authored the strategic AI-justification argument (Section 2.3).	docs/requirements/ai-boundary-matrix.md commit; prompt/reasoning notes in prompts/README.md.
Integration Lead YAWE IVAN	Designed the initial system architecture diagram and component-interaction narrative; scoped the RAG/tools/frontend interfaces for Week 3–4.	https://github.com/YaweIvan/RESOLV-HQ/blob/main/docs/architecture/initial-architecture.md (Assisted by Claude designtool)
QA & Security Lead	Authored the 8 user stories and 24 acceptance criteria; compiled the Week 1 risk register; verified the boundary matrix against the human-approval requirement.	docs/requirements/user-stories.md commit; risk register entry in docs/weekly-reports/week1.md.

6.5 GitHub Repository Structure

```
RESOLV-HQ/
├── README.md
├── .env.example
├── docs/
│   ├── requirements/
│   │   ├── project-charter.md
│   │   ├── ai-boundary-matrix.md
│   │   └── user-stories.md
│   ├── architecture/
│   │   └── initial-architecture.md
│   ├── weekly-reports/
│   │   └── week1.md
│   └── evaluation/
├── prompts/
│   └── README.md
├── knowledge/
│   └── provenance.md        # metadata only; no restricted data committed
├── src/
├── tests/
└── evidence/
    ├── traces/
    ├── screenshots/
    └── demo/
```

Figure 2. RESOLV-HQ GitHub repository structure (Week 1 baseline).

6.6 Plan for Next Week (Week 2)

- Select and document the foundation model (capability, cost, latency, privacy, access considerations).
- Integrate the model into a minimal working baseline interaction.
- Draft Prompt Specification v1.0 and begin the 10-case prompt evaluation table.